Tema 3. Confidencialidad de la Información (III)
Protocolos Seguros

Dr. Rafael Martínez Gasca
Grupo de Investigación **Quivir**,
Tecnologías Inteligentes y de Seguridad de los Sistemas de Información
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Sevilla





Indice

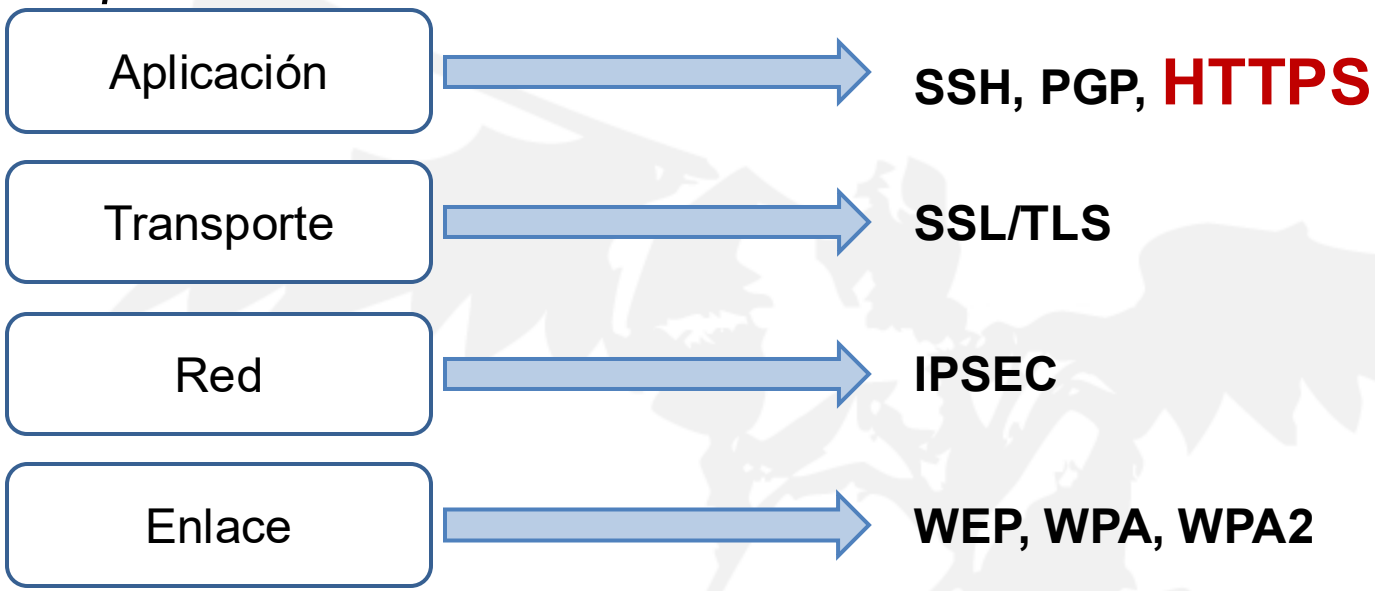
- Introducción. Conceptos Fundamentales
- Requisitos de la seguridad de la información
- Implementación tecnológica de los requisitos



- Técnicas Criptográficas
  - Cifrado de clave simétrica
  - Cifrado de clave asimétrica
  - Protocolo de Reparto de Clave
  - Protocolo SSL/TLS,SSH, PGP y SRTP
  - Protocolo WEP, WAP y WAP2
  - XML Encryption
- Esteganografía

Implementación de la Confidencialidad.
Transmisión. HTTPS

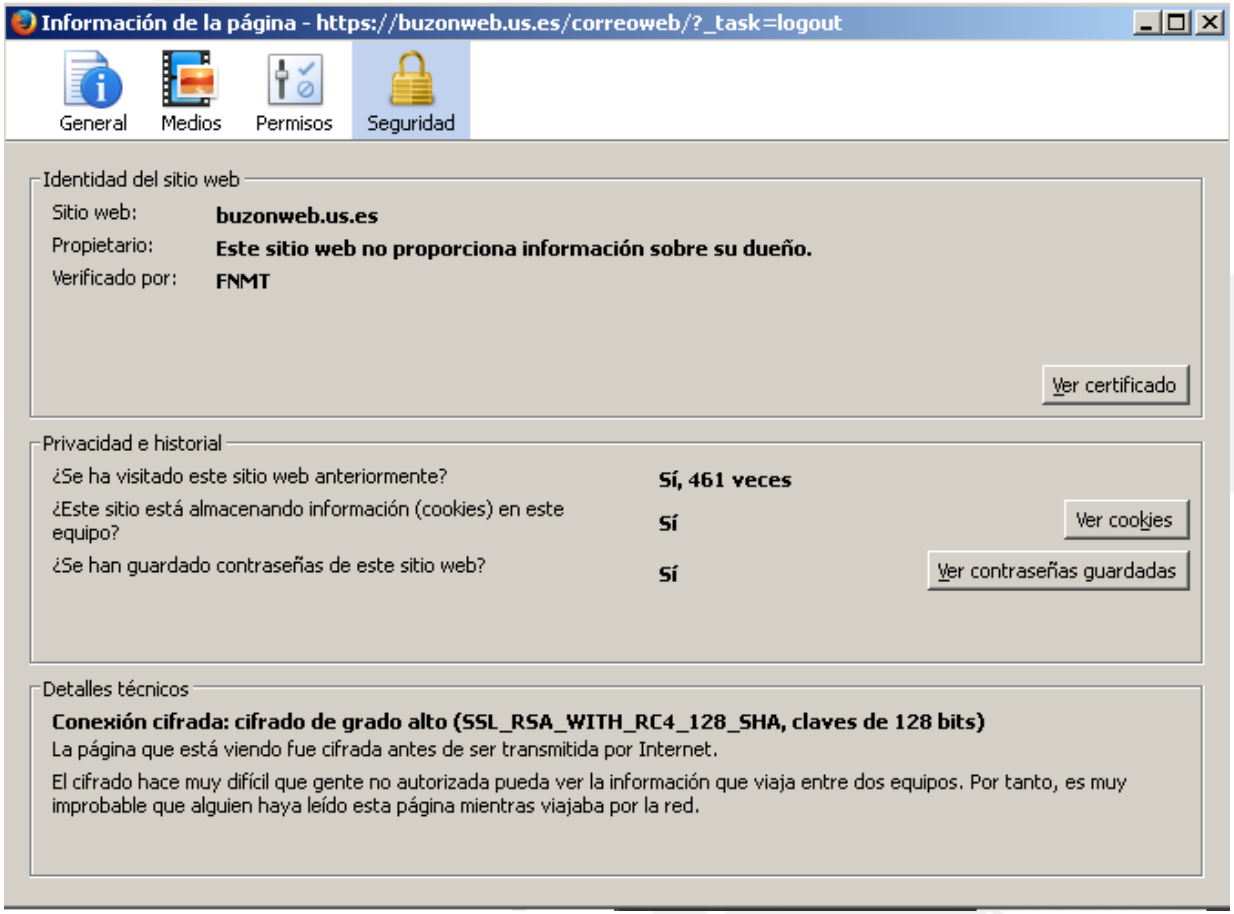
- *En los clientes Web para dar seguridad en transmisiones se usa https*



Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

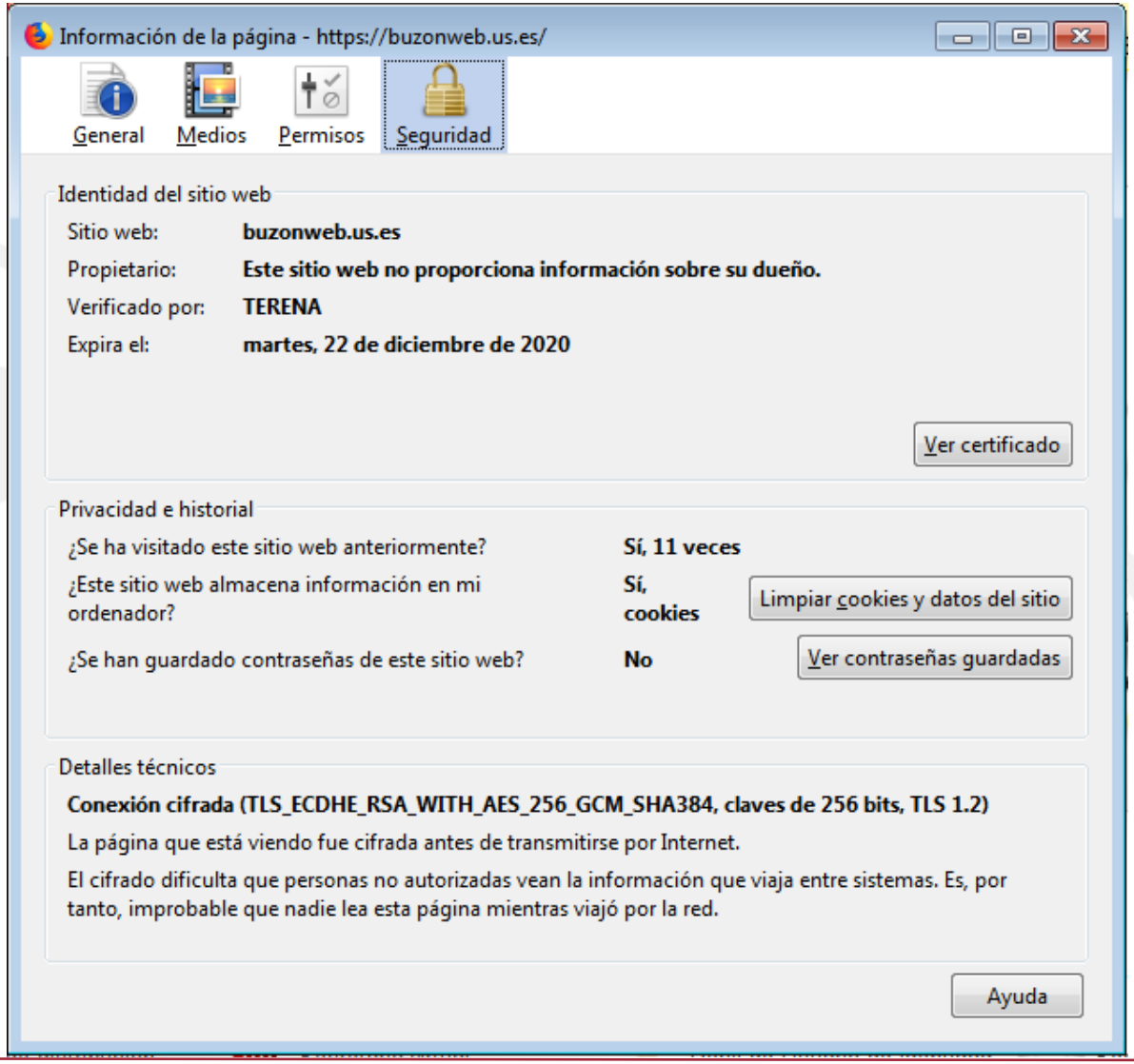
- *Introducido por el Navegador Netscape (Mozilla Firefox)*
- *Aplicación de SSL/TLS (Servidor requiere certificado y los clientes lo pueden tener o no) para comunicación*
- *Similar al protocolo HTTP pero por el puerto 443 (RFC 2812)*
- ***¿Cómo saber si una conexión HTTPS es segura?***
- *Qualys SSL Labs mantiene una colección de herramientas que ayuda a entender las conexiones seguras SSL/TLS.*
  - *Página para analizar tu cliente Web Auditar Mi Cliente Web*

- *Ejemplo de hace años https://buzonweb.us.es*



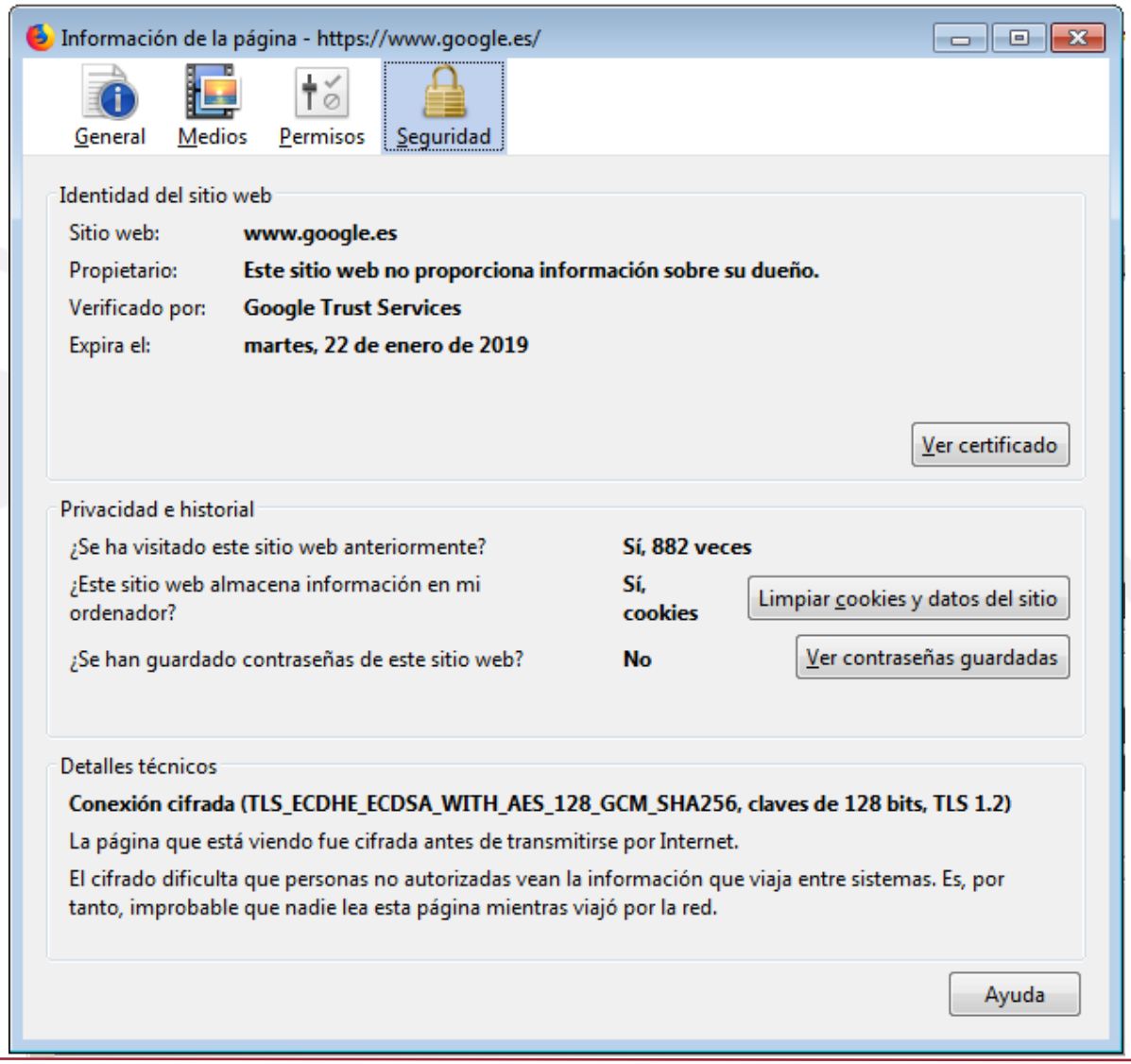
Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

- *Ejemplo actual:*



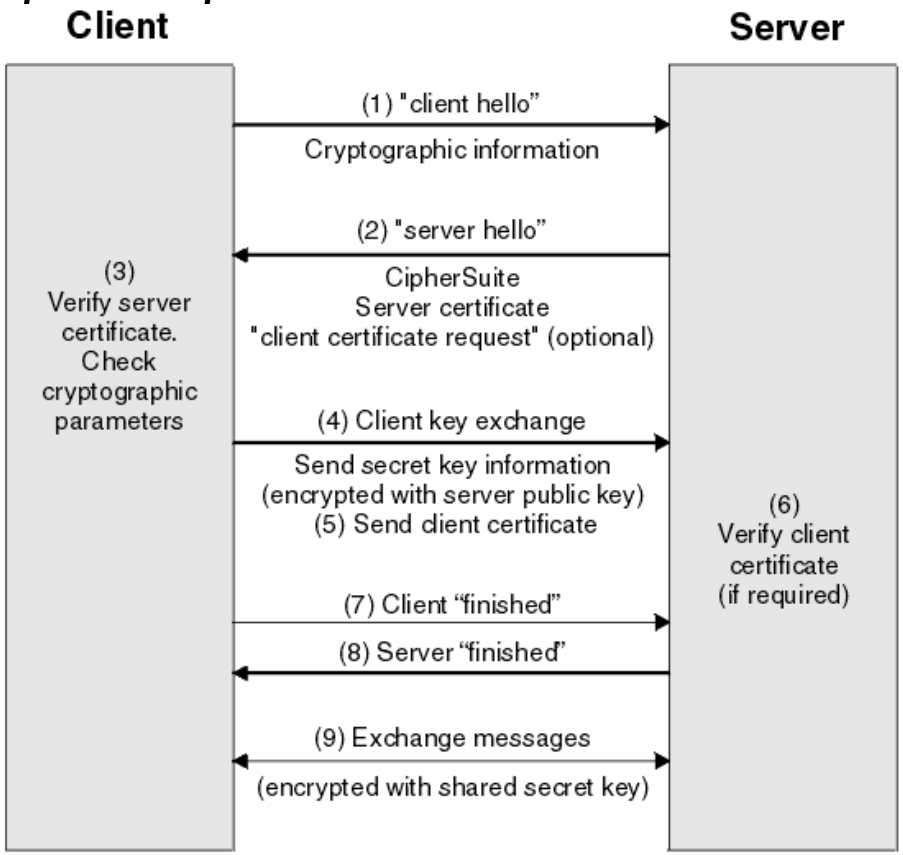
Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

- *Ejemplo actual:*



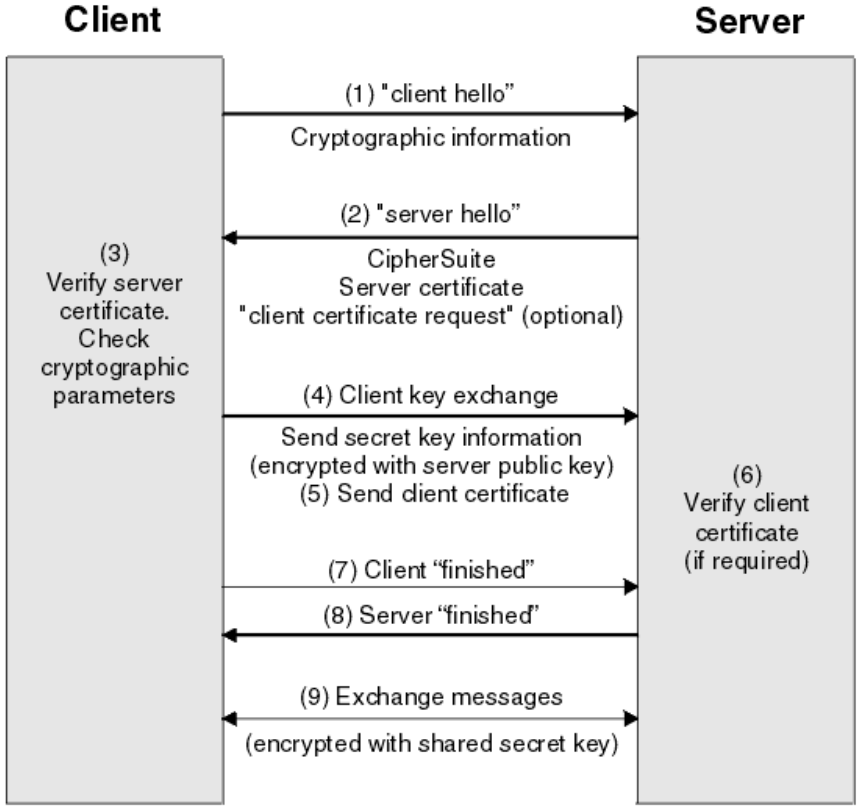
Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

- *TLS permite intercambiar información comprimida, cifrada e integra (código de autenticación del mensaje MAC).*
- *Cada comunicación tiene un campo de content_type que especifica el protocolo de nivel superior que se está usando*
- *Cuando se inicia la conexión se usa el protocolo de acuerdo (handshake)*



- *El cliente envía y recibe mensajes para el acuerdo (handshake):*
  1. *Envía un mensaje **ClientHello** especificando una lista de conjunto de cifrados, la versión del protocolo TLS más alta permitida. Puede incluir el identificador de la sesión anterior*
  2. *Después, recibe un mensaje **ServerHello**, en el que el servidor elige los parámetros de conexión (CipherSuite) a partir de las opciones del cliente. Y se incluye el certificado **X.509** del servidor (clave pública del servidor).*
  3. *El cliente verifica el certificado del servidor y también el cliente puede enviar su certificado X.509 (paso 5 y 6)*

*4. Cliente y servidor negocian una clave secreta (simétrica) común llamada (**Master Secret**): usando intercambio Diffie-Hellman o similar, o simplemente cifrando una clave secreta con una clave pública de la parte contraria. Todos las claves restantes serán derivadas de la Master Secret y los valores aleatorios intercambiados*



Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

- *La confidencialidad en el flujo de datos se mantiene con la clave secreta acordada*
- *La integridad se consigue con resúmenes MAC*
- *La autenticación se logra con los certificados X.509*
- *Protección contra varios ataques conocidos:*
  - *Degradado del protocolo a versiones previas (por tanto, menos seguras), o conjuntos de cifrados más débiles*
  - *Ataques man-in-the-middle. Se numeran todos los mensajes y se añade al resumen MAC. El mensaje que finaliza el protocolo handshake envía un resumen de todos los datos intercambiados y vistos por ambas partes*

Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

- **Forward secrecy**: *Es una propiedad de los sistemas criptográficos que garantiza que una clave de sesión derivada de un conjunto de claves públicas y privadas no se verá comprometida si una de las claves privadas se ve comprometida en el futuro*
- *Sin forward secrecy, si las sesiones pasadas fueran interceptadas y almacenadas en el momento de la transmisión podrían ser recuperadas*
- *Una implementación de TLS puede proporcionar forward secrecy al exigir el uso de intercambio* **Diffie-Hellman** *para establecer la clave de cifrado secreta*

- **Diffie-Hellman**:
  - *Cada interlocutor elige dos números públicos y un número secreto*
  - *Cada interlocutor hace una serie de operaciones con los dos números públicos y el secreto*
  - *Al final los interlocutores se intercambian los resultados*
  - *Cada uno utiliza por separado una fórmula matemática que combina los dos números transformados con su número secreto y al final los dos llegan al mismo número, que se usará como base de la clave que compartirán*
- *La clave compartida no es transmitida en el intercambio*

- *Algunas versiones de Diffie-Hellman usados en SSL/TLS.*
  - **Anonymous Diffie-Hellman**. *Usa Diffie-Hellman, pero sin autenticación. Las claves usadas en el intercambio no son autenticadas y por tanto* ***susceptibles de ataques Man-in-the-Middle.***
  - **Fixed Diffie-Hellman (DH)** *incorpora los parámetros públicos del servidor en el certificado firmado por uan CA. (el certificado contiene los parámetros de la clave pública de Diffie-Hellman, y éstos nunca cambian).*
  - **Ephemeral Diffie-Hellman (DHE)** *usa claves secretas temporales. Cada instancia o ejecución del protocolo usa diferente clave secreta. La autenticidad de la clave temporal del servidor se puede verificar por la firma de la clave. Debido a que las claves son temporales, un compromiso de las* ***claves firmadas actuales del servidor no pone en peligro la privacidad de pasadas sesiones*** *(****Perfect Forward Secrecy PFS).***

Implementación de la Confidencialidad.
Transmisión. HTTPS Protocol

- *Según el NIST la suite de seguridad en clientes/servidores Web son (para OpenSSL hay nombres parecidos):*
  - **RSA (RSA) key exchange**
  - **RSA with Ephemeral Diffie Hellman (DHE-RSA) key exchange**
  - **RSA with Elliptic Curve Ephemeral Diffie Hellman (ECDHE-RSA) key exchange**
  - **RSA with Elliptic Curve Diffie Hellman (ECDH-RSA) key exchange**
  - **ECDSA with Elliptic Curve Ephemeral Diffie Hellman (ECDHE-ECDSA) key exchange**
  - **ECDSA with Elliptic Curve Diffie Hellman (ECDH-ECDSA) key exchange**
  - **Pre Shared Key (PSK) key exchange**
  - **Pre Shared Key with Diffie Hellman (DHE-PSK) key exchange**
  - **Pre Shared Key with Elliptic Curve Ephemeral Diffie Hellman (ECDHE-PSK) key exchange**
  - **Pre Shared Key with RSA (RSA-PSK) key exchange**

- *Se pueden indicar los soportados en servidores/clientes Web*
  - *Uso de intercambio de claves RSA*
    - **TLS-RSA-WITH-AES-128-CBC-SHA**
    - **TLS-RSA-WITH-AES-256-CBC-SHA**
    - **TLS-RSA-WITH-AES-128-CBC-SHA256**
    - **TLS-RSA-WITH-AES-256-CBC-SHA256**
    - **TLS-RSA-WITH-AES-128-GCM-SHA256 (GCM Galois/Counter Mode)**
    - **TLS-RSA-WITH-AES-256-GCM-SHA384**
    - **TLS-RSA-WITH-CAMELLIA-128-CBC-SHA**
    - **TLS-RSA-WITH-CAMELLIA-256-CBC-SHA**
    - **TLS-RSA-WITH-CAMELLIA-128-GCM-SHA256**
    - **TLS-RSA-WITH-3DES-EDE-CBC-SHA**
    - **TLS-RSA-WITH-RC4-128-SHA**
    - **TLS-RSA-WITH-RC4-128-MD5**
    - **TLS-RSA-WITH-DES-CBC-SHA**

– *Uso de **RSA with Ephemeral Diffie Hellman (DHE-RSA**)*
  - **TLS-DHE-RSA-WITH-AES-128-CBC-SHA**
  - **TLS-DHE-RSA-WITH-AES-256-CBC-SHA**
  - **TLS-DHE-RSA-WITH-AES-128-CBC-SHA256**
  - **TLS-DHE-RSA-WITH-AES-256-CBC-SHA256**
  - **TLS-DHE-RSA-WITH-AES-128-GCM-SHA256**
  - **TLS-DHE-RSA-WITH-AES-256-GCM-SHA384**
  - **TLS-DHE-RSA-WITH-CAMELLIA-128-CBC-SHA**
  - **TLS-DHE-RSA-WITH-CAMELLIA-256-CBC-SHA**
  - ….

- *Uso de **RSA with Elliptic Curve Ephemeral Diffie Hellman (ECDHE-RSA)***
  - **TLS-ECDHE-RSA-WITH-AES-128-CBC-SHA**
  - **TLS-ECDHE-RSA-WITH-AES-256-CBC-SHA**
  - **TLS-ECDHE-RSA-WITH-AES-128-CBC-SHA256**
  - **TLS-ECDHE-RSA-WITH-AES-256-CBC-SHA384**
  - **TLS-ECDHE-RSA-WITH-AES-128-GCM-SHA256**
  - **TLS-ECDHE-RSA-WITH-CAMELLIA-128-CBC-SHA256**
  - **TLS-ECDHE-RSA-WITH-3DES-EDE-CBC-SHA**
  - **TLS-ECDHE-RSA-WITH-RC4-128-SHA**
  - **…..**

- *Uso de **Pre Shared Key with Diffie Hellman (DHE-PSK)*** **Request for Comments: 4279**
  - *TLS_PSK_WITH_RC4_128_SHA PSK RC4_128 SHA*
  - *TLS_PSK_WITH_3DES_EDE_CBC_SHA PSK 3DES_EDE_CBC SHA*
  - *TLS_PSK_WITH_AES_128_CBC_SHA PSK AES_128_CBC SHA*
  - *TLS_PSK_WITH_AES_256_CBC_SHA PSK AES_256_CBC SHA*
  - *TLS_DHE_PSK_WITH_RC4_128_SHA DHE_PSK RC4_128 SHA*
  - *TLS_DHE_PSK_WITH_3DES_EDE_CBC_SHA DHE_PSK DES_EDE_CBC SHA*
  - *…*
- *Cipher suite soportados por Java y OpenSSL respectivamente*
  - *http://docs.oracle.com/javase/7/docs/technotes/guides/security/SunProviders.html#SUNProvider*
  - *https://www.openssl.org/docs/apps/ciphers.html#CIPHER_SUITE_NAMES*

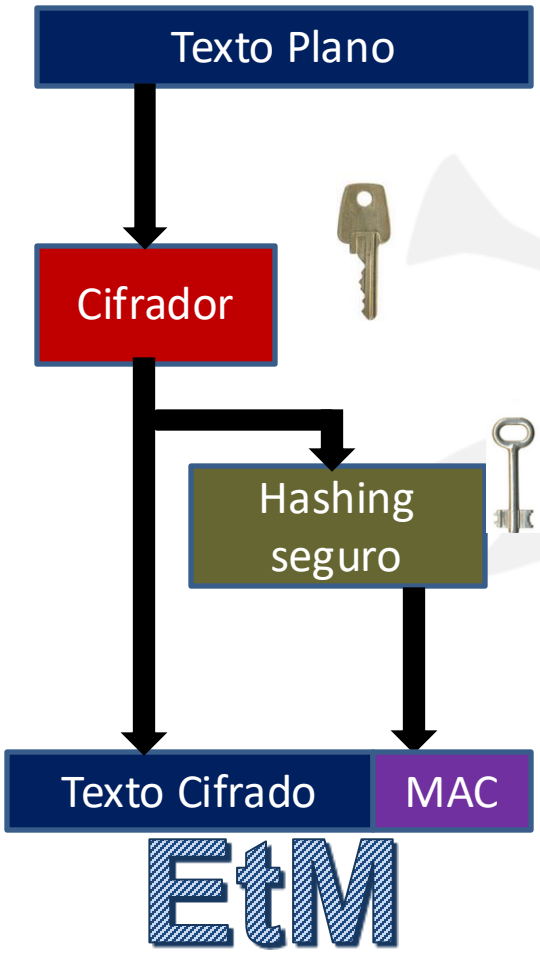
- **TLS 1.3** *publicada en Agosto de 2018 en RFC 8446.*
  - *Acelera las comunicaciones seguras gracias a su enfoque simplificado.*
  - *Permite el llamado "**0-RTT Resumption**" para que de forma efectiva cliente y servidor recuerden si hablaron antes, de tal forma que se renuncia a todas las comprobaciones, usándose claves previas para empezar a hablar. Un atacante para acceder a dichas claves lo tendría bastante difícil pues requeriría acceder a las máquinas*
  - **No se permiten versiones obsoletas de cifrado**

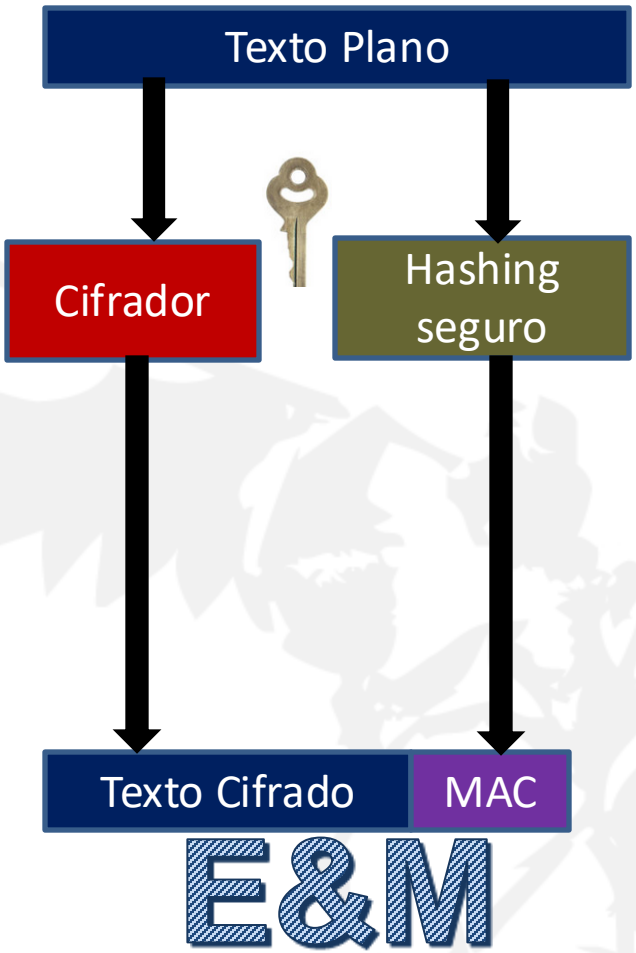
Implementación de la Confidencialidad.
Transmisión. SSL/TLS

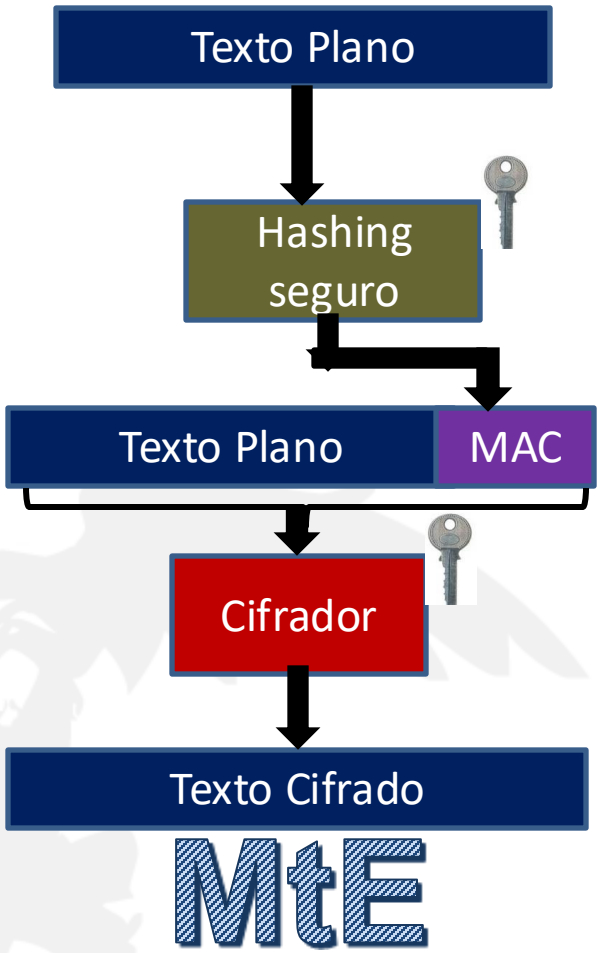


- **TLS 1.3** *vs. TLS 1.2: Más rápido, más seguro y más simple*
- **Mayor Velocidad***: Conexiones más rápidas y menor latencia*
  - *Reducción del Handshake: TLS 1.3 reduce el saludo inicial a 1 sola ida y vuelta (1-RTT), en comparación con los 2-RTT de TLS 1.2*
  - *Función 0-RTT (Zero Round-Trip Time): Permite a clientes que ya se han conectado antes enviar datos en el primer mensaje (¿seguro?)*
- **Seguridad Reforzada***: Eliminación de lo obsoleto*
  - *El handshake se cifra antes que en TLS 1.2*
  - *Desaparecen algoritmos vulnerables: SHA-1, RC4, DES, 3DES, MD5 y AES-CBC (propensos a ataques como POODLE)*
- **Simplicidad y Eficiencia**
  - *Menos Cipher Suites, se reduce a solo 5 suites de cifrado (basadas en AEAD)*

Implementación de la Confidencialidad.
Modos de cifrado







AEAD
Authenticated Encryption with Associated Data

TLS 1.3 obliga al uso de AEAD. Realiza el cifrado y la autenticación en un único paso

Implementación de la Confidencialidad.
Transmisión. SSL/TLS

- *TLS 1.3 el* **nombre** *de la cipher suite solo define el algoritmo de cifrado simétrico y el de integridad*
- *Los algoritmos de intercambio de claves y certificados se negocian en el handshake*
- *Las 5* **Cipher Suites** *de TLS 1.3:*
  - *TLS_AES_256_GCM_SHA384 (standard)*
  - *TLS_AES_128_GCM_SHA256 (muy rápido)*
  - *TLS_CHACHA20_POLY1305_SHA256 (dispositivos sin aceleración por hardware para AES)*
  - *TLS_AES_128_CCM_SHA256 (IoT o con recursos limitados)*
  - *TLS_AES_128_CCM_8_SHA256 (IoT de baja ancho de banda)*



Implementación de la Confidencialidad.
Transmisión. SSL/TLS

- *TLS 1.3 obliga a usar algoritmos con* **Perfect Forward Secrecy** *(PFS), para el* **intercambio de clave simétrica***:*
  - *DHE (Diffie-Hellman Ephemeral). Basada en aritmética modular de números primos, el cliente y el servidor generan un par de claves matemáticas totalmente nuevas y aleatorias para cada sesión*
  - *ECDHE (Elliptic Curve Diffie-Hellman Ephemeral): por ejemplo curvas X25519 o P-256 (estándar actual).*
    - *Se utilizan propiedades geométricas de una curva eliptica en lugar de artimética modular con números primos*
    - *Una clave de curva elíptica de 256 bits ofrece la misma resistencia que una clave DH tradicional de 3072 bits*
- *Algoritmos para* **Certificados y Firmas Digitales**
  - *RSA-PSS (añade un "salt" aleatorio en las firmas para mas seguridad)*
  - *ECDSA (Elliptic Curve Digital Signature Algorithm)*
  - *EdDSA (Ed25519 o Ed448, firmas rápidas basadas en curvas de Edwards)*



- *Implementaciones de TLS por terceros*
  - *Lenguajes y frameworks tales como Go, Swift, Java, rustls, and Apple's Network.framework incluye soporte para TLS 1.3.*
  - *Los más importantes browsers, incluyendo Chrome, Firefox, and Safari suportan TLS 1.3 por defecto*
  - *Las implementaciones más importantes de TLS son as NSS, BoringSSL, OpenSSL, GnuTLS, wolfSSL que soportan la nueva versión para TLS 1.3, además de las anteriores.*

- *Cambiar la configuración: Por ejemplo podrías desactivar el uso de RC4.*
  - **Usar la ruta la URL *about:config* aparece la página con una lista de preferencias y sus valores.**
  - **Se busca lo que se desea cambiar, por ejemplo escribir *rc4*. Con ello estamos filtrando las opciones. Haremos doble clic en cada una de las opciones a cambiar de true a false o al revés.**
  - **Cerramos el navegador podemos ver cómo el algoritmo de cifrado ha cambiado a otro.**
  - **http://www.kriptopolis.com/defensa-ssl-anular-rc4**

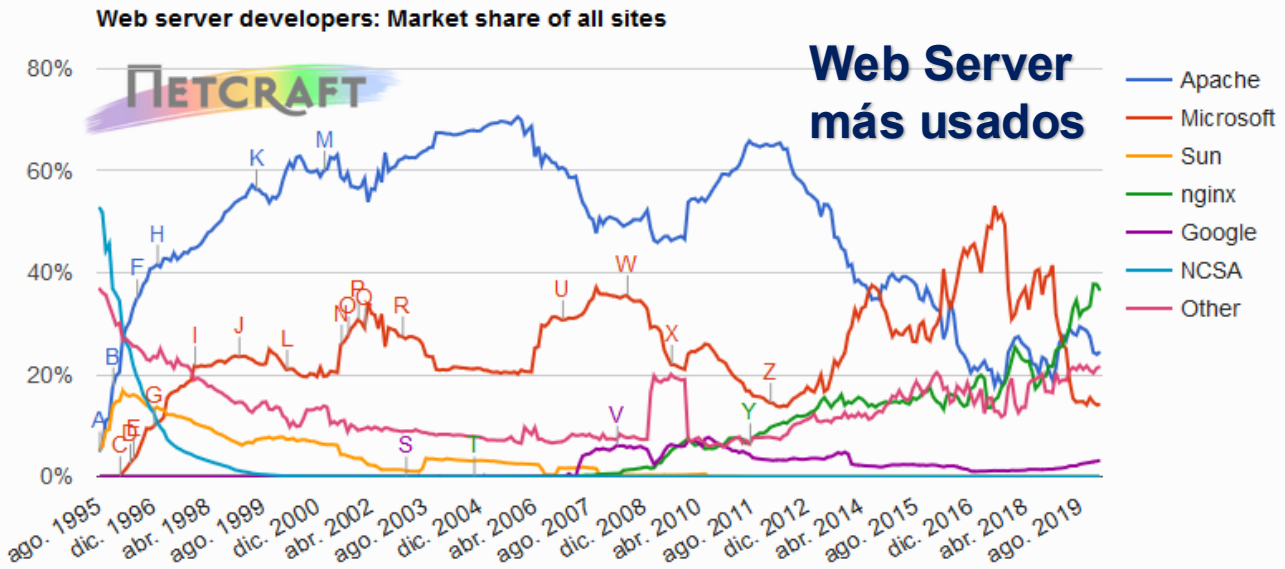
Test para débiles cipher-suite

http://foundeo.com/products/iis-weak-ssl-ciphers/test.cfm

- *Calidad del protocolo https*
  https://www.ssllabs.com/ssltest/index.html
  - *Diferentes categorías de calidad*
  - *Hay páginas que le sobra la s final de https*
  - **Testea los servidores Web y los navegadores Web**
  - *Muestra cómo están haciendo la seguridad Web otras empresas*
  - *La propia página se da una calificación*
- *Uso de favicon verde en la barra de navegación para la calidad de los certificados (certificados de validación extendida)*

- **Testing de seguridad de los clientes para el protocolo TLS**
  - *Qualys SSL Labs mantiene una colección de herramientas que ayuda a entender las conexiones seguras SSL/TLS.*
  - *Auditar tu cliente Web Auditar Mi Cliente Web*
- **Aseguramiento de los clientes para el protocolo TLS**
  - *Configuración de las versiones del protocolo TLS que se admiten*
  - *Configuración de los cipher-suite admitidos*
  - *Configuración de los certificados que serán entregables*
  - *Configuración de las autoridades de certificación confiables*
  - *Configuración de permitir OCSP o OCSP stapling*
  - *Configuración de no permitir compresión TLS*
  - *Configuración del contenido mixto activo o pasivo admitidos*

Implementación de la Confidencialidad.
Transmisión. Testing y Hardening de Web Server



Developer	January 2020	Percent	February 2020	Percent	Change
nginx	488,628,547	37.70%	459,966,569	36.48%	-1.22
Apache	310,833,084	23.98%	309,061,300	24.51%	0.53
Microsoft	181,873,181	14.03%	179,225,073	14.21%	0.18
Google	39,081,956	3.02%	40,120,733	3.18%	0.17

- *Para servidores Apache: El módulo **mod_ssl** ofrece TLS y se debe configurar*
  - *Las versiones de protocol TLS que permite*
  - *Los cipher-suite que permitirán*
  - *Los certificados que entregarán a los clientes*
  - *Los almacenes de claves que se usarán para firmar (por ejemplo en DHE)*